Open in Colab

Lab 3.2 Power meter data collection using Modbus protocol

2.1 Python programming for data collection from power meter

The entire schematic of the data communication using Modbus protocol in case of multiple masters (computers) and a single slave (power meter) is shown in Figure 12. In the lab, your computers (laptop and Raspberry Pi) are on the router network. The Modbus RTU to Modbus TCP converter (gateway) is also on the same network with a static IP address. The ethernet cable (RJ45) is used for connection between the router and the gateway. The data communication sequence to read data from power meter is as follows.

1. The computer (Raspberry Pi or laptop) requests read (data) holding registers using Modbus TCP protocol via the router to the gateway.
2. The gateway converts Modbus TCP packet frame to Modbus RTU data frame and sends it to the slave.
3. The slave responds to the request and sends Modbus RTU data frame to the gateway.
4. The gateway converts the received Modbus RTU data frame to Modbus TCP packet frame.
5. The gateway sends the converted packet to a specific IP host which is the computer requested data.
6. The computer receives Modbus TCP packet frame and interprets the packet to the data.

To do this, we will use Python and 'pymodbus' module. If you are interested in 'pymodbus' module in detail, please visit the website (https://pymodbus.readthedocs.io/en/latest/, available on Aug. 5, 2024).

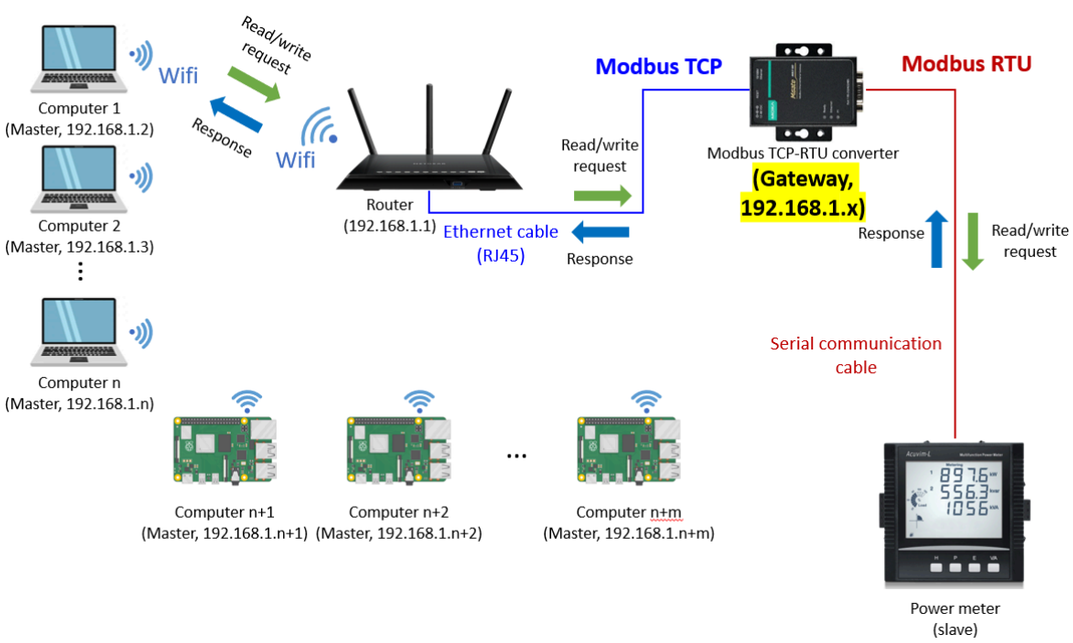


Figure 12 Schematic of communication for power meter data collection

What you need to do first to use TCP/IP communication and application is to check network connectivity. By using ping test from Raspberry Pi, check the network connection between Raspberry Pi and the gateway.

The IP address of the gateway at Lab3.
Offline class : 192.168.1.100
Online class : 10.165.67.146

Task 2.1

Capture the results of checking network connection using ping test from both Raspberry Pi and laptop to the gateway for Modbus TCP as Figure 13 and attach it to the report.

- Please note that the IP address of the gateway will be different from the example.

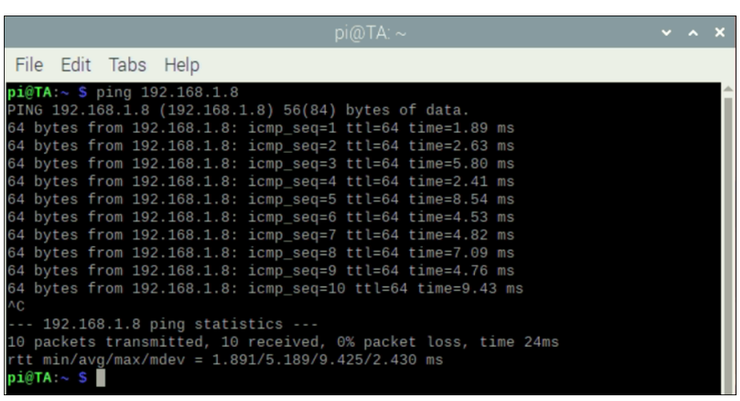
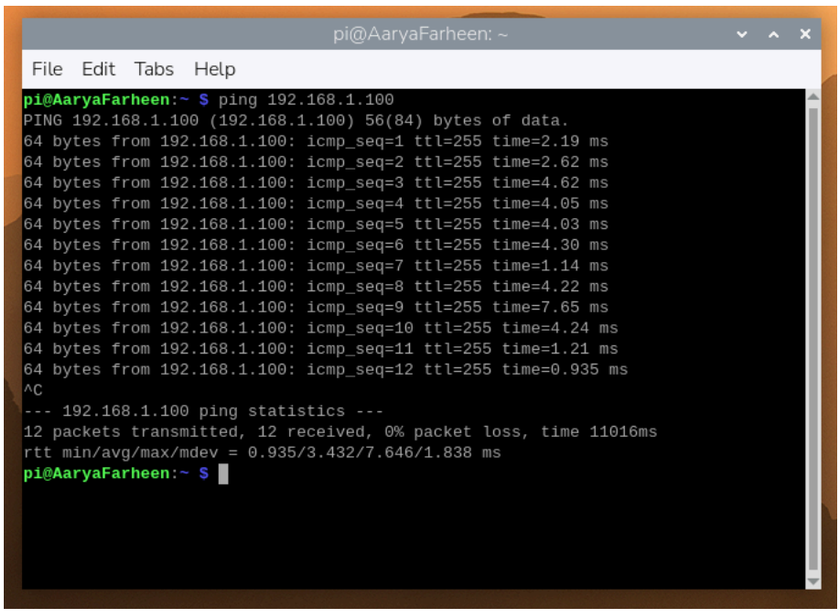
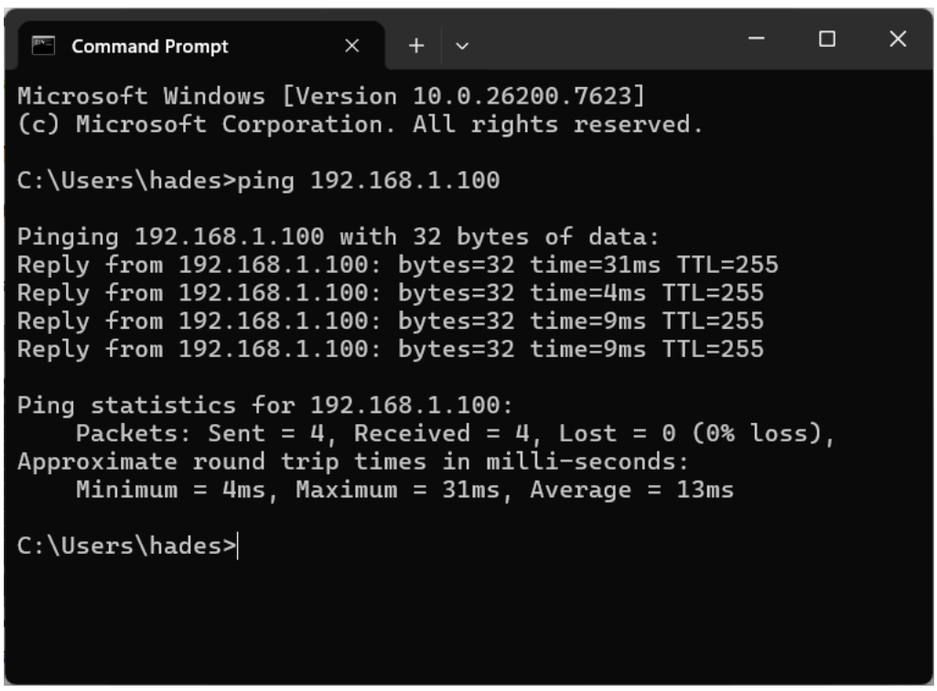


Figure 13 Terminal window after ping test from Raspberry Pi to the gateway





To read data from the power meter, we need to request to read registers via Modbus protocol. The selected primary data address of basic measurement of the power meter is shown in Table 2. There are hundreds of parameters (Modbus registers) that can be measured from the power meter. All Modbus map is on AccuEnergy website. Or refer to Table 5-16 on page 109 of the manual (Acuvim L User's Manual.pdf). Please note that the minimum refresh rate of holding registers of the power meter is about 250 msec which means you can collect 4 times per second. Therefore, your request interval should be larger than 250 msec.

Table 2 Selected primary data address of basic measurement of the power meter

Address (Hex)	Address (Dec	Symbol	Parameter	Range	Unit	Data Type	Access Property
0600H-0601H	1536-1537	F	Frequency	4500 ~ 6500	Hz	float	R
0602H-0603H	1538-1538	U1	Phase 1 Voltage	0 ~ 65535	V	float	R
060EH-060FH	1550-1551	I1	Total Phase A Current	0 ~ 65535	A	float	R
061CH-061DH	1564-1565	Psum	Total System power	-32768 ~ 32768	W	float	R
0624H-0623H	1572-1573	Qsum	Total Reactive power	-32768 ~ 32768	var	float	R
0626H-0627H	1574-1575	Ssum	Total Apparent Power	0 ~ 65535	VA	float	R
0628EH-062FH	1582-1583	PFsum	Total Power Factor	-1000 ~ 1000	-	float	R

Python - Python 3 (lab3_sample1.py)

```
from pymodbus.client.sync import ModbusTcpClient
from pymodbus.constants import Endian
from pymodbus.payload import BinaryPayloadDecoder
import datetime

host = "192.168.1.100"  # Online student with Cisco VPN: "10.165.67.146"
port = 502

client = ModbusTcpClient(host, port=port)
```

```
if not client.connect():
    raise RuntimeError(f"Connect failed: {host}:{port}")

now = datetime.datetime.now()

freq_read = client.read_holding_registers(1536, count=2, unit=1)

if freq_read.isError():
    client.close()
    raise RuntimeError(f"Modbus read error: {freq_read}")

freq_reg = freq_read.registers

decoder = BinaryPayloadDecoder.fromRegisters(
    freq_reg,
    byteorder=Endian.Big,
    wordorder=Endian.Big
)


freq_value = decoder.decode_32bit_float()

print(f"{now}: Frequency is {freq_value} Hz")

client.close()
```

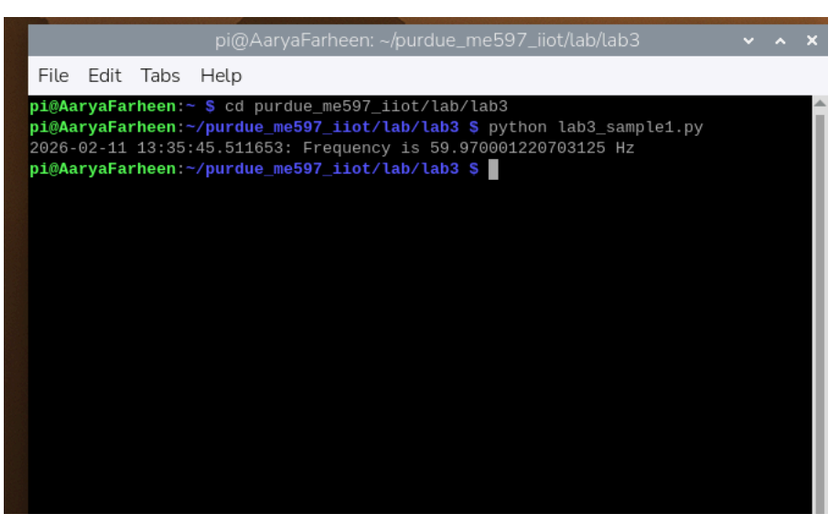
The sample code 1 (lab3_sample1.py) is given as above. This code is to print out a timestamp and the measured frequency from the power meter as Figure 12. As expected, the measured frequency is 60 Hz. Please refer to the comments on the script for the descriptions. Please note that the read values of registers by Modbus protocol are encoded and it may be differed by the device. In this case, the register values are decoded by Big Endian Byte order. For the details of understanding Byte order encoding and decoding, please visit the website (https://www.digital-detective.net/understanding-big-and-little-endian-byte-order/, available on Feb. 1, 2023).

Task 2.2

Run the sample code 1 and then capture and attach the result as Figure 14.

- Capture either one of terminal window as Figure 14 (top) or Thonny IDE Shell as Figure 14 (bottom) from Raspberry Pi.
- Try this on your laptop and see if it works as well.
- (Online class only) If you have trouble connecting network router, review L0_Colab1 1.9 Section.

Figure 14 Results of running sample code 1: terminal window (top) and Thonny IDE shell (bottom)



Task 2.3

1. Modify sample code 1 to collect power meter data for at least 1 minute and save data as a CSV file.

- You can use one computer either laptop or Raspberry Pi.
- Hint: Make a function (method) to convert received registers to a float value. An input argument is a set of registers read and the output is a float value.
- Set the sampling period 1 second. (Hint: use ‘while’ loop and ‘time’ package)
- Data (header) should include time, frequency, voltage, current, power factor, and true power as Table 3. Note that you have to make sure the unit of each data.
- Refer to the sample codes in previous lab manuals ('while' loop and saving CSV file).
- As a practice, plot data and calculate time-domain features.

Table 3 Example data of output CSV file for Task 2.3

Time [sec]	Frequency [Hz]	Voltage 1 [V]	Current 1 [A]	Power factor [-]	True power [W]
			...		
30.17739	60.0008443	119.73915	2.481679	0.744922	221.9443
31.204921	60.0007994	120.011049	2.452804	0.737972	221.4492
			...		

```
### Place your entire code for Task 2.3 here
# If you want, you can add more code below to load csv file and plot it.
from pymodbus.client.sync import ModbusTcpClient
from pymodbus.constants import Endian
from pymodbus.payload import BinaryPayloadDecoder
import time
import csv
import numpy as np
import matplotlib.pyplot as plt

# ------------------------------------------------
# Function to convert Modbus registers to float
# ------------------------------------------------
def registers_to_float(registers):
    decoder = BinaryPayloadDecoder.fromRegisters(
        registers,
        byteorder=Endian.Big,
        wordorder=Endian.Big
    )
    return decoder.decode_32bit_float()

# ------------------------------------------------
# Modbus Configuration
# ------------------------------------------------
host = "192.168.1.100"   # Change if needed
port = 502
unit_id = 1

client = ModbusTcpClient(host, port=port)

if not client.connect():
    raise RuntimeError(f"Connection failed: {host}:{port}")

print("Connected to Power Meter...")

# ------------------------------------------------
# Data Storage
# ------------------------------------------------
time_list = []
freq_list = []
volt_list = []
curr_list = []
pf_list = []
power_list = []

# ------------------------------------------------
# Data Collection (1 minute, 1 sec sampling)
# ------------------------------------------------
duration = 60  # seconds
start_time = time.time()

while (time.time() - start_time) < duration:

    elapsed_time = int(time.time() - start_time)

    # Read registers (Modify addresses if different in your meter)
    freq_read  = client.read_holding_registers(1536, 2, unit=unit_id)
    volt_read  = client.read_holding_registers(1538, 2, unit=unit_id)
    curr_read  = client.read_holding_registers(1550, 2, unit=unit_id)
    pf_read    = client.read_holding_registers(1564, 2, unit=unit_id)
    power_read = client.read_holding_registers(1582, 2, unit=unit_id)

    if not freq_read.isError():

        freq_value  = registers_to_float(freq_read.registers)
        volt_value  = registers_to_float(volt_read.registers)
        curr_value  = registers_to_float(curr_read.registers)
        pf_value    = registers_to_float(pf_read.registers)
        power_value = registers_to_float(power_read.registers)
```

```
        # Store values
        time_list.append(elapsed_time)     # Time [sec]
        freq_list.append(freq_value)       # Frequency [Hz]
        volt_list.append(volt_value)       # Voltage [V]
        curr_list.append(curr_value)       # Current [A]
        pf_list.append(pf_value)           # Power factor [-]
        power_list.append(power_value)     # True power [W]

        print(f"Time:{elapsed_time}s | "
              f"F:{freq_value:.2f} Hz | "
              f"V:{volt_value:.2f} V | "
              f"I:{curr_value:.2f} A | "
              f"PF:{pf_value:.2f} | "
              f"P:{power_value:.2f} W")

    time.sleep(1)

client.close()
print("Data collection completed.")

# ---------------------------------------------------
# Save Data to CSV
# ---------------------------------------------------
with open("power_meter_data.csv", mode="w", newline="") as file:
    writer = csv.writer(file)

    writer.writerow([
        "Time [sec]",
        "Frequency [Hz]",
        "Voltage 1 [V]",
        "Current 1 [A]",
        "Power factor [-]",
        "True power [W]"
    ])

    for i in range(len(time_list)):
        writer.writerow([
            time_list[i],
            freq_list[i],
            volt_list[i],
            curr_list[i],
            pf_list[i],
            power_list[i]
        ])

print("CSV file saved successfully.")

# ---------------------------------------------------
# Convert to NumPy Arrays
# ---------------------------------------------------
freq_arr = np.array(freq_list)
volt_arr = np.array(volt_list)
curr_arr = np.array(curr_list)
power_arr = np.array(power_list)

# ---------------------------------------------------
# Time-Domain Feature Calculation
# ---------------------------------------------------
print("\n----- Time Domain Features -----")

print("Voltage Mean:", np.mean(volt_arr))
print("Voltage RMS:", np.sqrt(np.mean(volt_arr**2)))
print("Voltage Std Dev:", np.std(volt_arr))

print("Current Mean:", np.mean(curr_arr))
print("Current RMS:", np.sqrt(np.mean(curr_arr**2)))
print("Current Std Dev:", np.std(curr_arr))

print("True Power Mean:", np.mean(power_arr))
print("True Power Std Dev:", np.std(power_arr))

# ---------------------------------------------------
# Plot Data
# ---------------------------------------------------
plt.figure()
plt.plot(time_list, volt_arr)
plt.title("Voltage vs Time")
plt.xlabel("Time [sec]")
plt.ylabel("Voltage 1 [V]")
plt.show()

plt.figure()
plt.plot(time_list, curr_arr)
plt.title("Current vs Time")
plt.xlabel("Time [sec]")
plt.ylabel("Current 1 [A]")
plt.show()

plt.figure()
plt.plot(time_list, power_arr)
plt.title("True Power vs Time")
plt.xlabel("Time [sec]")
plt.ylabel("True power [W]")
plt.show()
```

Show hidden output

Please continue to Lab 3.3 here.